National University of Singapore

SCHOOL OF COMPUTING

CS2030 — PROGRAMMING METHODOLOGY II
(Semester 2: AY2025/2026)

Apr / May 2026 Time Allowed: 2 Hours

INSTRUCTIONS TO CANDIDATES

1. This assessment contains **TWENTY-SEVEN(27)** questions.
2. Answer **ALL** questions. The maximum mark is **80**.
3. This is an **EXAMPLIFY-SECURE CLOSED-BOOK** assessment.
4. You will be liable for disciplinary action which may result in expulsion from the University if you are found to have contravened any of the clauses below,
   - Violation of the NUS Code of Student Conduct (in particular the part on Academic, Professional and Personal Integrity), NUS IT Acceptable Use Policy or NUS Examination rules.
   - Possession of unauthorized materials/electronic devices.
   - Bringing your mobile phone or any storage/communication device with you to the washroom.
   - Unauthorized communication e.g. with another student.
   - Reproduction of any exam materials outside of the exam venue.
   - Photography or videography within the exam venue.
   - Plagiarism, giving or receiving unauthorised assistance in academic work, or other forms of academic dishonesty.
5. Once you have completed the assessment,
   - Click on the "Exam Controls" button and choose "Submit Exam".
   - Check off "I am ready to exit my exam" and click on "Exit" to upload your answers to the server.
   - You will see a green confirmation window on your screen when the upload is successful. Please keep this window on your screen.
   - If you do not see a green window, please disconnect and reconnect your WiFi and try again.
   - Please be reminded that it is your responsibility to ensure that you have uploaded your answers to the Software.
6. A separate Examplify exam attachment containing the Java APIs for commonly used libraries in CS2030 is given separately.

SECTION A (15 Multiple Choice Questions : 30 Marks)
Section A comprises 15 multiple choice questions with five possible answers each. The last option, None of the above, should be chosen if the answer cannot be found in the first four choices. You may assume that all program fragments are syntactically correct and compilable, and that any deviations detected are due to either minor typos or short-cuts and abbreviations used. 2 marks for each correct answer; no penalty for wrong answer.

1. The class `Cell` below uses `Object` internally. A student claims it is *equivalent* to `Cell<T>` with a generic type parameter. Which scenario **proves the student wrong**?

```
class Cell {
    private final Object t;

    Cell(Object t) {
        this.t = t;
    }

    Object get() {
        return t;
    }
}
```

   (A) You can store a `String` in `Cell` but not in `Cell<String>`.

   (B) `new Cell("hi").get().length()` compiles without a cast.

   (C) `new Cell("hi").get().length()` fails to compile because `get()` returns `Object`, whereas `Cell<String>.get()` returns `String`, so no cast is needed.

   (D) `Cell` and `Cell<T>` both perform runtime type checking of stored values.

   (E) None of the above.

2. A student expects the following code to print `"expensive called"` only when `opt` is empty. What actually happens?

```
String expensive() {
    System.out.println("expensive called");
    return "default";
}

Optional<String> opt = Optional.of("value");
String result = opt.orElse(expensive());
```

(A) "`expensive called`" is never printed because `opt` is non-empty.

(B) "`expensive called`" is always printed regardless of whether `opt` is empty, because Java evaluates all method arguments eagerly before the call.

(C) A compile error occurs because `orElse` requires a literal.

(D) "`expensive called`" is printed only at the second invocation.

(E) None of the above.

3. The composition law of the functor states:

   `ctx.map(f).map(g)` $\equiv$ `ctx.map(g.compose(f))`

   Why does this law **matter in practice**?

   (A) It guarantees `map` is faster when two functions are composed first.

   (B) It guarantees chaining multiple `map` calls is semantically equivalent to composing the functions first, enabling refactoring and potential optimizations such as fusing traversals.

   (C) It guarantees `map` can only be called once per context.

   (D) It proves `map` and `flatMap` are interchangeable.

   (E) None of the above.

4. Given the following program fragment.

```
class A {
    void foo(A v) {
        System.out.print("1");
    }
}

class B extends A {
    void foo(A v) {
        System.out.print("2");
    }

    void foo(B v) {
        System.out.print("3");
    }
}

B b = new B(); A a = b;
b.foo(b);
a.foo(b);
a.foo(a);
```

What is the output?

(A) 322

(B) 323

(C) 312

(D) 332

(E) None of the above.

5. Which property **distinguishes** `flatMap` from `map`, for a given `Context<R>`?

   (A) `flatMap` can change the element type; `map` cannot.

   (B) `flatMap` accepts a function returning a `Context<R>` and avoids double-wrapping.

   (C) `flatMap` is always lazy; `map` is always eager.

   (D) `flatMap` is less general than `map`. (A method `m1` is less general than another method `m2`, if `m1` can be implemented in terms of `m2`.)

   (E) None of the above.

6. The following stream pipeline

```
stream.parallel().reduce(1, (x, y) -> x / y);
```

gives wrong results in parallel. Why?

(A) Division is not commutative.

(B) Division is not associative, so splitting the stream changes the result.

(C) `reduce` does not support parallel streams.

(D) The identity element `1` is invalid for division.

(E) None of the above.

7. Which usage of `Optional` is considered **undisciplined** in CS2030?

   (A) `opt.map(f)`

   (B) `opt.orElse(defaultVal)`

   (C) `if (opt.isPresent()) { opt.get(); }`

   (D) `opt.flatMap(f)`

   (E) None of the above.

8. Given the following program fragment.

```
Log<Integer> addTwo(int x) {
    return Log.of(x+2, "addTwo");
}

Log<Integer> multThree(int x) {
    return Log.of(x*3, "multThree");
}

Log.of(2).flatMap(x -> addTwo(x)).flatMap(r -> multThree(r));
```

What will be printed to represent the resulting `Log<Integer>` object?

(A) `Log[12]: addTwo, multThree`

(B) `Log[12]: multThree, addTwo`

(C) `Log[6]: addTwo, multThree`

(D) `Log[8]: addTwo, multThree`

(E) None of the above.

9. Given the following program fragment.

```
Lazy<Integer> v = Lazy.of(() -> { System.out.print("A"); return 5; });
Lazy<Integer> w = v.map(x -> x * 2);
System.out.print("B");
int result = w.get();
System.out.print("C");
```

What is the output?

(A) ABC

(B) BAC

(C) BAC5

(D) BAAC

(E) None of the above.

10. The left identity monad law states: `unit(a).flatMap(f) == f(a)`. Which `Optional` expression **demonstrates** this law?

    (A) `Optional.of(5).flatMap(x -> Optional.of(x + 1)) == Optional.of(6)`

    (B) `Optional.of(5).flatMap(x -> Optional.of(x)) == Optional.of(5)`

    (C) `Optional.empty().flatMap(f) == Optional.empty()`

    (D) `Optional.of(5).map(x -> x + 1) == Optional.of(6)`

    (E) None of the above.

11. Given the following program fragment.

```
CompletableFuture<Integer> cf = CompletableFuture.supplyAsync(() -> {
    System.out.print("running");
    return 42;
});

System.out.print("main");
cf.join();
```

Which output is **possible**?

(A) mainrunning

(B) runningmain

(C) Both (A) and (B) are possible.

(D) Neither — `supplyAsync` is synchronous.

(E) None of the above.

12. Given the following program fragment.

```
class Circle {
    private final Point centre;
    private final double radius;
}

class FilledCircle extends Circle {
    private final Color color;
}
```

Which statements are correct?

(i) `Circle` has-a `Point`;

(ii) `FilledCircle` is-a `Circle`;

(iii) `Circle` is-a `Point`;

(iv) `FilledCircle` has-a `Circle`.

(A) (i) only.

(B) (i) and (ii).

(C) (ii) and (iv).

(D) (i), (ii), and (iv).

(E) None of the above.

13. Which scenario calls for an **abstract class** rather than an interface?

    (A) You want `Circle` and `Rectangle` to both promise they have a `getArea()` method.

    (B) You want `FilledCircle` and `FilledRectangle` to share a stored `Color` field and a concrete `getColor()` method, while requiring each to implement `getArea()` differently.

    (C) You want `Circle` to be both a `Shape` and a `Movable`.

    (D) You want to specify that `Comparator<T>` has a `compare` method.

    (E) None of the above.

14. Given the following program fragment.

```
class Circle implements Shape {
    boolean equals(Circle c) { return ..; }   // overloaded

    @Override
    public boolean equals(Object o) { return ..; }  // overriding
}

Shape s = new Circle(1.0);
Circle c = new Circle(1.0);
s.equals(c); // (i)
c.equals(c); // (ii)
```

Which `equals` method is called for (i) and (ii)?

(A) (i) `Object.equals`; (ii) overloaded `Circle.equals(Circle)`.

(B) (i) overriding `Circle.equals(Object)`; (ii) overloaded `Circle.equals(Circle)`.

(C) (i) overriding `Circle.equals(Object)`; (ii) overriding `Circle.equals(Object)`.

(D) (i) `Object.equals`; (ii) overriding `Circle.equals(Object)`.

(E) None of the above.

15. Compare the following *imperative* program fragment

```
// Imperative:
int findCoverage(Optional<Circle> occ, Stream<Point> pts) {
    if (occ.isEmpty()) {
        return 0;
    }
    Circle cc = occ.get();
    return pts.filter(p -> cc.contains(p))
              .map(p -> 1)
              .reduce(0, (x, y) -> x + y);
}
```

with an alternative *declarative* program fragment below:

```
// Declarative:
int findCoverage(Optional<Circle> occ, Stream<Point> pts) {
    return occ.map(cc -> pts.filter(p -> cc.contains(p))
                            .map(p -> 1)
                            .reduce(0, (x, y) -> x + y))
  .orElse(0);
}
```

Which statement best explains why the *declarative* version is preferred?

(A) The declarative version is faster at runtime.

(B) The declarative version avoids `isEmpty()` and `get()`, which break `Optional`'s encapsulation by exposing the internal value.

(C) The declarative version works with `null` values safely.

(D) The imperative version throws `NoSuchElementException`.

(E) None of the above.

SECTION B (12 Questions : 50 Marks)
Unless otherwise specified, do not define your own classes or use classes other than those specified in the Java API, even if they have been defined in the course such as `Pair`, `Maybe`, `InfList`, `Lazy`, `Try`, `Log`, etc. You do not need to write import statements.

The following problem statement is for questions Q(16) to Q(18). You are advised to read through all the questions before attempting.

An airline operations team wants to analyse flight delay records. You are given the following `record` class and a pre-populated list:

```
record Flight(
    String flightId,   // e.g. "SQ321"
    String origin,     // IATA airport code, e.g. "SIN"
    String dest,       // IATA airport code, e.g. "LHR"
    int    delay,      // in minutes; negative = arrived early
    double fare        // ticket price in SGD
) {}

List<Flight> flights = List.of(
    new Flight("SQ321", "SIN", "LHR",  45, 1200.0),
    new Flight("SQ322", "SIN", "LHR",  -5,  980.0),
    new Flight("TR108", "SIN", "BKK",  20,  150.0),
    new Flight("TR109", "SIN", "BKK", 120,  130.0),
    new Flight("SQ801", "SIN", "SYD",   0,  750.0),
    new Flight("MI234", "SIN", "KUL", -10,   85.0),
    new Flight("SQ011", "SIN", "JFK",  60, 1800.0),
    new Flight("TR200", "SIN", "BKK",  35,  140.0),
    new Flight("SQ802", "SIN", "SYD",  15,  800.0),
    new Flight("MI235", "SIN", "KUL",   5,   90.0)
);
```

A flight is *delayed* if `delay > 0`, and *significantly delayed* if `delay > 30`.

16. **[3 marks]** Write a *single* `Stream` pipeline that prints, in the order they appear in the list, the flight IDs of *all significantly delayed flights* (`delay > 30`), each on a new line. Your pipeline must start with:

    ```
    flights.stream()
    ```

    Hint:

    - Use at most *two operations*: one intermediate and one terminal.
    - Do not use `null` or imperative `if`-statements outside a lambda.
    - Expected output (in order):

      ```
      SQ321
      TR109
      SQ011
      TR200
      ```

    ANSWER:

17. [**5 marks**] Write a Stream pipeline that, for each *distinct destination airport*, computes the *average fare* and prints destinations whose average fare *exceeds SGD 500*.

    In your solution,

    - first define a helper method:

      `double findAverageFare(List<Flight> flights, String dst)`

      that returns the average fare of all flights to `dst`, or `0.0` if no flights exist for that destination.
    - next using a `Pair` and a single pipeline starting with `flights.stream()`, print all destinations where the average fare exceeds `SGD 500`. The corresponding output should be:

      ```
      Pair[t=JFK, u=1800.0]
      Pair[t=LHR, u=1090.0]
      Pair[t=SYD, u=775.0]
      ```

    Hint:

    - Use `Pair` to pair destination with average.
    - For each `Pair pr`, filter it where `pr.u() > 500.0` and print with `forEach`;
    - You may find `DoubleStream`'s `average()` method useful.

    ANSWER:

18. [**4 marks**] The operations team wants a utility method that, given a destination airport code, finds *the single most expensive delayed flight to that destination.*

    Complete the method below using a *single* `Stream` pipeline with *exactly one* `return` statement, and with ≤ 6 stream operations inside the method body.

```
Optional<Flight> mostExpensiveDelayed(
        List<Flight> flights, String dest) {
    return flights.stream()...
}
```

    The pipeline must:

    i. filter for flights to `dest`;

    ii. retain only delayed flights (`delay > 0`);

    iii. return the highest-fare flight wrapped in `Optional`.

    The following are **strictly forbidden**:

    - `null`
    - any methods that checks/exposes the encapsulated value within the `Optional`, e.g. `isPresent()`, `isEmpty()`, `get()`, etc.
    - any loop

    ANSWER:

The following problem statement is for questions Q(19) to Q(22). You are advised to read through all the questions before attempting.

Relationships between people can be represented as a graph where each node is a person and each directed edge represents the a person liking another person. A *person* can be characterized as either an *extrovert* or an *introvert*. Extroverts actively go around liking other people, while an introvert only waits for others to like him. Below is an example of relationship graph.

```
jshell> Introvert D = new Introvert("D") // a graph starts with an introvert
D ==> Introvert: D

jshell> Extrovert B = new Extrovert("B", D) // extrovert likes at least one person
B ==> Extrovert: B

jshell> Extrovert A = new Extrovert("A", B)
A ==> Extrovert: A

jshell> Extrovert E = new Extrovert("E", B)
E ==> Extrovert: E

jshell> Extrovert C = new Extrovert("C", E)
C ==> Extrovert: C

jshell> A = A.likes(C).likes(E) // extrovert can like more people
A ==> Extrovert: A

jshell> D.likes(A) // introvert does not like anyone
|  Error:
|  cannot find symbol
|    symbol:   method likes(Extrovert)
|  D.likes(A)
|  ^-----^

jshell> D instanceof Person
$.. ==> true

jshell> B instanceof Person
$.. ==> true

jshell> new Person("P")
|  Error:
|  Person is abstract; cannot be instantiated
|  new Person("P")
|  ^-------------^
```

Design the `Person`, `Introvert` and `Extrovert` classes while adhering to the guidelines:

- all class properties **must** be `private`;
- all objects **must** be immutable;
- adherance to the DRY (don't repeat yourself) principle.

Use the `InfList` to store the friends that a person likes.

19. [**4 marks**] Write the `Person` class.

ANSWER:

20. **[3 marks]** Write the `Introvert` class.

ANSWER:

21. [**5 marks**] Write the `Extrovert` class. You will also need to include an overloaded `likes()` method that takes no arguments to output the list of friends. The ordering of friends does not matter.

```
jshell> A.likes().forEach(x -> System.out.println(x))
Extrovert: B
Extrovert: C
Extrovert: E

jshell> D.likes()
|  Error:
|  cannot find symbol
|    symbol:   method likes()
|  D.likes()
|  ^-----^
```

Note from the last test that the `likes()` method cannot be invoked from `Introvert`.

ANSWER:

22. [**5 marks**] Within the `Extrovert` class, include the method `hasCycle` that takes in the relationships represented in `graph` and returns `true` if there is a cycle detected in `graph`, and `false` otherwise.

```
boolean hasCycle(InfList<Person> graph) {
   ...
}
```

Using the same example,

```
jshell> Introvert D = new Introvert("D")
D ==> Introvert: D

jshell> Extrovert B = new Extrovert("B", D)
B ==> Extrovert: B

jshell> Extrovert A = new Extrovert("A", B)
A ==> Extrovert: A

jshell> Extrovert E = new Extrovert("E", B)
E ==> Extrovert: E

jshell> Extrovert C = new Extrovert("C", E)
C ==> Extrovert: C

jshell> A = A.likes(C).likes(E)
A ==> Extrovert: A

jshell> A.hasCycle(InfList.of(A, B, C, D, E))
$.. ==> false
```

In the above, starting from person `A` and traversing through all other reachable people, there is no cycle. However, by adding `B` likes `E`

```
jshell> B = B.likes(E)
B ==> Extrovert: B

jshell> A.hasCycle(InfList.of(A, B, C, D, E))
$.. ==> true
```

There is now a cycle since `E` likes `B` and `B` likes `E`.

Hint: a *cycle* occurs when following the "likes" relationship starting from a person eventually leads back to someone already visited. In the latter graph above, starting from `A`, the path $A \to E \to B \to E$ revisits E, forming a cycle. In contrast, in the preceding example, no path starting from A ever revisits a person: $A \to B \to D$ (dead end), $A \to C \to E \to B \to D$ (dead end), so `hasCycle` returns `false`.

ANSWER:

The following problem statement is for questions Q(23) to Q(27). You are advised to read through all the questions before attempting.

This is a follow up to the `IStack<T>` class that you designed from the recent practical assessment. However, rather than a class, `IStack` is now an abstract class with a correct concrete subclass of `IStack`, such as `IStackImpl` (not given).

```
abstract class IStack<T> {
    // factory method
    static <T> IStack<T> empty() { // instantiates an empty stack
       return new IStackImpl<T>();
    }

    // toString method
    public String toString() {
        return "IStack:" + (this.isEmpty() ? "empty" : "non-empty");
    }

    // core operations
    abstract IStack<T> push(T t); // returns new stack with t on top

    abstract IStack<T> pop(); // returns new stack with top element removed

    abstract Maybe<T> peek(); // returns Maybe of top element; or
                              // Maybe.empty() if stack is empty

    abstract boolean isEmpty(); // returns true if stack is empty

    abstract void print(); // outputs the stack in LIFO order
}
```

In the following questions, you are to complete the specified methods within the `IStack` class. Answers for each question may include methods defined in preceding questions.

Important Note: These questions need not be completed in sequence. For example, if you were unable to complete the `fold` method of question Q(26), you can still use `fold` to implement the `contains` method that was asked in question Q(27).

23. [**4 marks**] Complete the following `pushAll` method that takes in another stack `other`, so that every element that is popped from `other` is pushed into the current stack.

```
IStack<T> pushAll(IStack<? extends T> other) {
    return ...
}
```

For example,

```
jshell> IStack<Integer> s1 = IStack.<Integer>empty().push(1).push(2).push(3)
s1 ==> IStack:non-empty

jshell> IStack<Integer> s2 = IStack.<Integer>empty().push(4).push(5).push(6)
s2 ==> IStack:non-empty

jshell> s1.pushAll(s2).print()
4
5
6
3
2
1
```

Note: *Your answer need not include the method header, unless it is a helper method.*

ANSWER:

24. [**3 marks**] Using the preceding `pushAll` method, complete the `reverse()` method that reverses the contents of the current stack.

```
IStack<T> reverse() {
    return ...
}
```

For example,

```
jshell> IStack.<Integer>empty().
   ...> push(1).
   ...> push(2).
   ...> push(3).
   ...> reverse().
   ...> print()
1
2
3
```

Note: *Your answer need not include the method header, unless it is a helper method.*

ANSWER:

25. [**5 marks**] `IStack<T>` should now support `map`. Complete the `map` method that takes in a function `mapper` and applies the function to every element of the stack, while preserving the LIFO (last-in-first-out) order, i.e. elements from the top of the original stack must appear at the top of the resulting stack.

```
<R> IStack<R> map(Function<? super T, ? extends R> mapper) {
    return ...
}
```

For example,

```
jshell> IStack.<Integer>empty().
   ...> push(1).
   ...> push(2).
   ...> push(3).
   ...> map(x -> "[" + x + "]").print()
[3]
[2]
[1]
```

Note: *Your answer need not include the method header, unless it is a helper method.*

ANSWER:

26. [**5 marks**] We now want to add a `fold` operation that collapses a `IStack<T>` into a single value of type `T`. Implement the following method **inside** class `IStack<T>`:

    ```
    Maybe<T> fold(BinaryOperator<T> op)
    ```

    such that it combines all elements of the stack using `op`, starting from the top, and returns `Maybe.empty()` if the stack is empty. `fold` is **right-associative**: the top element is the leftmost operand and is combined with the result of folding the rest:

    ```
    IStack[1, 2, 3, 4].fold(op) = Maybe[op(1, op(2, op(3, 4)))]
    ```

    This differs from stream's

    ```
    Optional<T> Stream.reduce(BinaryOperator<T> op)
    ```

    which is left-associative:

    ```
    Stream.of(1, 2, 3, 4).reduce(op) = Optional[op(op(op(1, 2), 3), 4)]
    ```

    Here are some examples:

    ```
    jshell> IStack<Integer> stack = IStack.<Integer>empty().
       ...> push(1).
       ...> push(2).
       ...> push(3).
       ...> push(4)
    stack ==> IStack:non-empty

    jshell> stack.fold((x, y) -> x + y)
    $.. ==> Maybe[10]

    jshell> stack.fold((x, y) -> x - y)
    $.. ==> Maybe[-2]

    jshell> stack.fold((x, y) -> x * y)
    $.. ==> Maybe[24]
    ```

    Constraints:

    - Use only `peek`, `pop`, `isEmpty` and recursion — no loops, no streams.
    - The result must be wrapped in `Maybe<T>`, and you may use the methods of the `Maybe` class.
    - Do not use any imperative mutation.

    ANSWER:

27. [**4 marks**] Complete the method `contains` that takes in an element `t` and returns `true` if `t` is present in the stack, and `false` otherwise (including for an empty stack).

```
boolean contains(T t) {
    return ...
}
```

For example,

```
jshell> IStack<Integer> stack = IStack.<Integer>empty().
   ...> push(1).
   ...> push(2).
   ...> push(3).
   ...> push(4)
stack ==> IStack:non-empty

jshell> stack.contains(3)
$.. ==> true

jshell> stack.contains(5)
$.. ==> false

jshell> IStack.<Integer>empty().contains(5)
$.. ==> false
```

Note: *Your answer need not include the method header, unless it is a helper method.*

ANSWER: